

CleverFerret Android App - Comprehensive Fix Proposals

Generated: October 14, 2025

Repository: /home/ubuntu/CleverFerret

Analysis Scope: Compilation errors, TODOs, dependency issues, and architectural concerns

Executive Summary

The CleverFerret Android app is a comprehensive universal media library application with ambitious features including EPUB reading, comic translation, Jellyfin/Plex integration, and AI-powered content analysis. The codebase shows good architectural patterns (MVVM with Hilt DI, Room database, Jetpack Compose UI) but currently has **critical compilation blockers** preventing the app from building successfully.

Critical Statistics

- **33 compilation errors** across 5 key files
 - **95 TODO/FIXME markers** indicating incomplete implementations
 - **1 critical Hilt configuration error** preventing builds
 - Multiple stub/simplified implementations marked for future completion
-

Priority Classification

CRITICAL - Must Fix to Build

1. Hilt dependency injection configuration error
2. Jellyfin SDK integration (API changes)
3. Radium EPUB service (Try API changes)
4. ComicProcessorRepository (Google Play Services Tasks)
5. EnhancedEReaderScreen (Compose scope issues)
6. EpubReaderEngine (scope resolution)

HIGH - Code Quality & Functionality

1. 95 TODO markers requiring implementation
2. Stub implementations in Jellyfin, Plex, Emby services
3. Missing dependency imports (kotlinx.coroutines.tasks)

MEDIUM - Enhancements & Polish

1. Architecture improvements
 2. Error handling standardization
 3. Documentation gaps
-

Section 1: Critical Compilation Errors

1.1 Hilt Configuration Error (BLOCKER)

Location: Build system

Error:

```
Execution failed for task ':CleverFerret:hiltAggregateDepsDebug'
> Unexpected annotation value: rootComponentPackage
```

Root Cause: Version mismatch between Hilt Gradle plugin and Hilt Android library

- Gradle plugin: 2.51.1

- Hilt Android: 2.57.2

Impact: Build fails completely - nothing can compile

Proposed Fix:

```
// In build.gradle.kts (root level)
buildscript {
    dependencies {
        // Update Hilt Gradle plugin to match library version
        classpath("com.google.dagger:hilt-android-gradle-plugin:2.57.2") // Changed
        from 2.51.1
    }
}
```

Alternative Fix (if above doesn't work):

```
// In CleverFerret/build.gradle.kts
dependencies {
    // Downgrade Hilt to match plugin
    implementation("com.google.dagger:hilt-android:2.51.1") // Changed from 2.57.2
    ksp("com.google.dagger:hilt-compiler:2.51.1")
}
```

Recommendation: Use the first fix (upgrade plugin) as 2.57.2 is newer and has bug fixes.

1.2 ComicProcessorRepository - Missing Tasks Extension

Location: services/comic/ComicProcessorRepository.kt:14, 203, 227

Errors:

```
Line 14: Unresolved reference 'tasks'
Line 203: Unresolved reference 'await'
Line 227: Unresolved reference 'await'
Line 264: Argument 'type' mismatch (ByteArray vs Bitmap)
```

Current Code:

```
import kotlinx.coroutines.tasks.await // Line 14 - This import fails

// Line 203
translator.downloadModelIfNeeded(conditions).await()

// Line 227
translator.translate(text).await()
```

Root Cause: The package `kotlinx.coroutines.tasks` doesn't exist in the standard coroutines library. The correct extension comes from `kotlinx-coroutines-play-services`.

Proposed Fix:

```
// CORRECT import
import kotlinx.coroutines.tasks.await // This should come from play-services

// Verify dependency in build.gradle.kts
dependencies {
    // This provides the .await() extension for Google Play Services Tasks
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0") //
    Already present
}
```

Additional Investigation Needed:

The dependency is already in `build.gradle.kts`. This suggests:

1. Gradle sync issue - try invalidating caches
2. KSP processing order - tasks extension may not be visible during compilation
3. Import conflict - another library may be shadowing this

Alternative Workaround (if above doesn't resolve):

```
import com.google.android.gms.tasks.Tasks
import kotlinx.coroutines.suspendCancellableCoroutine
import kotlin.coroutines.resume
import kotlin.coroutines.resumeWithException

// Create custom await extension
suspend fun <T> com.google.android.gms.tasks.Task<T>.await(): T = suspendCancellableC-
oroutine { cont ->
    addOnSuccessListener { cont.resume(it) }
    addOnFailureListener { cont.resumeWithException(it) }
    addOnCanceledListener { cont.cancel() }
}
```

Line 264 Type Mismatch Fix:

```
// Current (WRONG)
val bitmap = generativeModel.generateContent(content)
// This returns ByteArray but Bitmap expected

// Fixed
val imageBytes = generativeModel.generateContent(content)
val bitmap = BitmapFactory.decodeByteArray(imageBytes, 0, imageBytes.size)

// Add import
import android.graphics.BitmapFactory
```

1.3 Jellyfin SDK Integration Issues

Location: services/integration/jellyfin/JellyfinClient.kt:34-40

Errors:

Line 34: None of the following candidates **is** applicable **for** Jellyfin constructor
 Line 34-40: Multiple unresolved references (clientInfo, httpClientOptions, client, name, version)

Current Code:

```
val optionsBuilder = JellyfinOptions.Builder().apply {
    this.clientInfo = clientInfo
    this.deviceInfo = deviceInfo
    this.context = null // Android context not required for API client
}

jellyfin = Jellyfin(optionsBuilder.build())
```

Root Cause: The Jellyfin SDK 1.5.4 API changed. The `JellyfinOptions.Builder` properties and initialization methods have been updated.

Proposed Fix:

```

/**
 * Initialize connection to Jellyfin server
 */
fun initialize(serverUrl: String, apiKey: String? = null) {
    try {
        // Create Jellyfin instance with updated API
        val clientInfo = ClientInfo(
            name = "CleverFerret",
            version = "1.0.0"
        )

        val deviceInfo = DeviceInfo(
            id = UUID.randomUUID().toString(),
            name = "Android Device"
        )

        // Use the correct constructor
        jellyfin = Jellyfin {
            // Configure client options
            this.clientInfo = clientInfo
            this.deviceInfo = deviceInfo
        }

        apiClient = if (apiKey != null) {
            jellyfin?.createApi(
                baseUrl = serverUrl,
                accessToken = apiKey
            )
        } else {
            jellyfin?.createApi(baseUrl = serverUrl)
        }
    } catch (e: Exception) {
        Log.e(TAG, "Failed to initialize: ${e.message}", e)
    }
}

```

Alternative Approach (if above doesn't work, check SDK docs):

```

// Jellyfin SDK 1.5.4 may use different initialization
val jellyfin = Jellyfin(
    JellyfinOptions(
        clientInfo = ClientInfo("CleverFerret", "1.0.0"),
        deviceInfo = DeviceInfo(
            id = UUID.randomUUID().toString(),
            name = "Android Device"
        )
    )
)

```

Documentation Reference Needed:

Check the Jellyfin Kotlin SDK 1.5.4 changelog and initialization examples:

- <https://github.com/jellyfin/jellyfin-sdk-kotlin>
- Specifically look for `JellyfinOptions` and `Jellyfin` constructor changes

1.4 Radium EPUB Service - Try API Changes

Location: `services/epub/RadiumEpubService.kt:48-58`

Errors:

```
Line 48: Conflicting declarations (asset defined twice)
Line 48: 'when' expression must be exhaustive
Line 48-58: Multiple 'Unresolved reference assetResult'
Line 50, 57: Type arguments expected for Try.Failure and Try.Success
```

Current Code:

```
val assetResult = assetRetriever.retrieve(file)
val asset = when (assetResult) {
    is Try.Success -> assetResult.value
    is Try.Failure -> {
        Log.e(TAG, "Failed to retrieve asset: ${assetResult.value}")
        return@withContext null
    }
}
```

Root Cause: Radium 3.1.2 changed their `Try` result type structure. The API may now use a different result type or require type parameters.

Proposed Fix - Option 1 (Add type parameters):

```

suspend fun extractCover(epubPath: String): Bitmap? = withContext(Dispatchers.IO) {
    try {
        val file = File(epubPath)
        if (!file.exists()) {
            Log.e(TAG, "EPUB file not found: $epubPath")
            return@withContext null
        }

        // Retrieve the asset from the file
        val asset = when (val assetResult = assetRetriever.retrieve(file)) {
            is Try.Success<Asset, *> -> assetResult.value
            is Try.Failure<*, AssetRetriever.Error> -> {
                Log.e(TAG, "Failed to retrieve asset: ${assetResult.value}")
                return@withContext null
            }
        }

        val publication = when (val parseResult = epubParser.parse(asset)) {
            is Try.Success<Publication.Builder, *> -> parseResult.value.build()
            is Try.Failure<*, EpubParser.Error> -> {
                Log.e(TAG, "Failed to parse EPUB: ${parseResult.value}")
                return@withContext null
            }
        }

        val coverBitmap = extractCoverFromPublication(publication)
        publication.close()
        coverBitmap

    } catch (e: Exception) {
        Log.e(TAG, "Error extracting cover: ${e.message}", e)
        null
    }
}

```

Proposed Fix - Option 2 (Use when expression with variable):

```

suspend fun extractCover(epubPath: String): Bitmap? = withContext(Dispatchers.IO) {
    try {
        val file = File(epubPath)
        if (!file.exists()) {
            Log.e(TAG, "EPUB file not found: $epubPath")
            return@withContext null
        }

        // Retrieve asset
        val assetResult = assetRetriever.retrieve(file)
        if (assetResult is Try.Failure) {
            Log.e(TAG, "Failed to retrieve asset: ${assetResult.value}")
            return@withContext null
        }
        val asset = (assetResult as Try.Success).value

        // Parse EPUB
        val parseResult = epubParser.parse(asset)
        if (parseResult is Try.Failure) {
            Log.e(TAG, "Failed to parse EPUB: ${parseResult.value}")
            return@withContext null
        }
        val publication = (parseResult as Try.Success).value.build()

        val coverBitmap = extractCoverFromPublication(publication)
        publication.close()
        coverBitmap

    } catch (e: Exception) {
        Log.e(TAG, "Error extracting cover: ${e.message}", e)
        null
    }
}

```

Recommended: Check Radium 3.1.2 documentation for the correct `Try` type signature:

```

// Expected type signature (verify with docs)
sealed class Try<out V, out E> {
    data class Success<V>(val value: V) : Try<V, Nothing>()
    data class Failure<E>(val value: E) : Try<Nothing, E>()
}

```

1.5 EnhancedEReaderScreen - Compose Scope Issues

Location: `ui/reader/EnhancedEReaderScreen.kt:296-319`

Errors:

```

Line 296, 303: Unresolved reference 'chapters'
Line 303: Cannot infer type 'for' parameter (2 occurrences)
Line 304, 313: @Composable invocations can only happen from context of @Composable function
Line 308: Unresolved reference 'goToChapter'
Line 319: Overload resolution ambiguity

```


Root Cause: Multiple issues:

1. Missing `chapters` property in scope
2. Lambda called outside `@Composable` context
3. Missing navigation callback

Investigation Needed:

Need to see the full function context to understand the scope. This appears to be a callback or event handler trying to access Compose state incorrectly.

Likely Fix Pattern:

```
// WRONG - Composable called in regular callback
onClick = {
    chapters.forEach { chapter -> // chapters undefined
        Text(chapter.title) // Composable invocation outside @Composable
        viewModel.goToChapter(chapter.index) // goToChapter undefined
    }
}

// CORRECT - Use ViewModel and state
@Composable
fun ChapterSelector(
    chapters: List<Chapter>,
    onChapterSelected: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(modifier = modifier) {
        items(chapters) { chapter ->
            Text(
                text = chapter.title,
                modifier = Modifier.clickable {
                    onChapterSelected(chapter.index)
                }
            )
        }
    }
}

// In parent composable
val viewModel: ReaderViewModel = hiltViewModel()
val chapters by viewModel.chapters.collectAsState()

ChapterSelector(
    chapters = chapters,
    onChapterSelected = { index -> viewModel.navigateToChapter(index) }
)
```

Specific Fix Recommendation:

1. Review lines 290-320 for full context
2. Move Composable UI into proper `@Composable` functions
3. Use `ViewModel` methods for navigation
4. Pass state as parameters instead of accessing in callbacks

1.6 EpubReaderEngine - Content Reference

Location: `services/epub/EpubReaderEngine.kt:389`

Error: Unresolved reference 'content'

Context (from lines 385-395):

```
val content = zipFile.getInputStream(entry).bufferedReader().readText()

chapters.add(
    EpubChapter(
        index = index,
        title = title,
        content = content, // Line 389 - Error here
        resourceId = itemRef
    )
)
```

Root Cause Analysis:

This error is puzzling because `content` is clearly defined 4 lines above. Possible causes:

1. Scope issue - the variable is defined in a different scope (e.g., inside a conditional that's not visible at line 389)
2. Compiler cache issue
3. Type inference failure causing subsequent errors

Investigation Required:

Need to see lines 370-400 with proper scope markers to understand the actual structure.

Likely Fix (if scope issue):

```

private fun extractChaptersFromSpine(
    zipFile: ZipFile,
    spineItems: List<String>,
    manifestItems: Map<String, ManifestItem>,
    tocItems: List<TocItem>
): List<EpubChapter> {
    val chapters = mutableListOf<EpubChapter>()
    val opfDir = findContentOpf(zipFile)?.name?.substringBeforeLast("/") ?: ""

    spineItems.forEachIndexed { index, itemRef ->
        val manifestItem = manifestItems[itemRef]
        manifestItem?.let { item -> // Use let to ensure proper scope
            val fullPath = if (opfDir.isNotEmpty()) "$opfDir/${item.href}" else item.h
            ref
            val entry = zipFile.getEntry(fullPath) ?: zipFile.getEntry(item.href)

            entry?.let { zipEntry -> // Nested let for entry
                val title = tocItems.find { it.href.contains(item.href) }?.title
                    ?: "Chapter ${index + 1}"

                val content = zipFile.getInputStream(zipEntry).bufferedReader().use {
                    it.readText() }

                chapters.add(
                    EpubChapter(
                        index = index,
                        title = title,
                        content = content, // Now definitely in scope
                        resourceId = itemRef
                    )
                )
            }
        }
    }

    return chapters
}

```

Alternative: If the original code structure is correct, try:

```

# Clean and rebuild
./gradlew clean
./gradlew :CleverFerret:compileDebugKotlin --rerun-tasks

```

Section 2: Dependency Issues Analysis

2.1 Current Dependencies Overview

Core Libraries (Well-Configured):

-  Kotlin 2.0.20
-  Compose BOM 2025.10.00
-  Hilt 2.57.2 (library) / 2.51.1 (plugin) 
-  Room 2.8.2
-  Navigation Compose 2.9.5
-  Retrofit 2.11.0

Media Libraries:

-  Radium Kotlin Toolkit 3.1.2
-  Media3 1.8.0
-  ExoPlayer (via Media3)

Integration SDKs:

-  Jellyfin SDK 1.5.4 (API incompatibility)
-  Google Cast Framework 22.1.0
-  Google AI Generative AI 0.9.0

Special Libraries:

-  ML Kit Translate 17.0.3
-  Play Services Tasks (import issues)
-  JUnrar 7.5.5 (Comic CBR support)

2.2 Dependency Conflicts

Hilt Version Mismatch (CRITICAL)

```
Plugin: 2.51.1
Library: 2.57.2
Impact: Build failure
```

Fix: Align versions (see Section 1.1)

AndroidX Legacy Support

Multiple libraries show warnings about mixing AndroidX and old support library:

- play-services-cast-framework
- mediarouter
- media
- core

Current Workaround: Jetifier is enabled

Recommendation: These are warnings, not errors. Can be addressed later.

```
// In gradle.properties (already present)
android.useAndroidX=true
android.enableJetifier=true
```

Play Services Tasks Extension

Issue: `kotlinx.coroutines.tasks.await` import fails despite dependency being present

Diagnostic Commands:

```
# Verify dependency is downloaded
./gradlew :CleverFerret:dependencies --configuration debugCompileClasspath | grep play
-services

# Check for conflicts
./gradlew :CleverFerret:dependencies --configuration debugCompileClasspath | grep task
s
```

Possible Resolutions:

1. Explicit dependency addition (may be transitive currently)
2. Clean build and reimport
3. Check Kotlin compiler version compatibility

Section 3: TODO Markers & Incomplete Code

3.1 TODO Statistics

Total TODOs: 95 across the codebase

By Category:

- Media Integration: 15 TODOs
- Metadata & Duration: 12 TODOs
- Watch History & Progress: 10 TODOs
- TTS & Audio: 8 TODOs
- File Streaming: 6 TODOs
- OCR & Translation: 5 TODOs
- Other: 39 TODOs

3.2 Critical TODOs (Affect Core Functionality)

Media Player Service

File: `services/media/UniversalMediaPlayerService.kt`

```
// TODO: Add ExoPlayer support when dependencies are available
```

Impact: Media playback may not work properly

Priority: HIGH

Recommendation: ExoPlayer is available via Media3 dependency. Implement ExoPlayer-based playback.

Media Session Duration

File: `services/media/MediaSessionManager.kt`

```
// TODO: Set duration for MediaMetadata - need to check Media3 API
```

Impact: Media controls won't show duration/progress

Priority: HIGH

Fix:

```
val mediaMetadata = MediaMetadata.Builder()  
    .setTitle(title)  
    .setArtist(artist)  
    // Add duration using Media3 API  
    .setDurationMs(durationMs)  
    .build()
```

Media Notification

File: `services/media/MediaNotificationService.kt`

```
// TODO: Get MediaSession from proper source
// TODO: Initialize artworkLoader properly when this feature is implemented
// TODO: Investigate proper Media3 notification integration with MediaNotificationManager
```

Impact: Media notifications may not display correctly

Priority: HIGH

Recommendation: Implement proper MediaNotificationManager integration with MediaSession

3.3 Watch History & Progress Tracking

Affected Files:

- `services/playlist/MoviePlaylistManager.kt`
- `services/playlist/TVShowPlaylistManager.kt`
- `services/playlist/AudiobookPlaylistManager.kt`

Common Pattern:

```
watched = false, // TODO: Integrate with watch history
progress = 0f, // TODO: Integrate with progress tracking
rating = 0f // TODO: Integrate with rating system
```

Impact: User progress and watch history not tracked

Priority: MEDIUM

Recommendation:

1. Create `WatchHistoryManager` service
2. Create `ProgressTrackingManager` service
3. Integrate with Room database for persistence
4. Update playlist managers to use these services

3.4 TTS Providers

File: `services/tts/TtsProviderManager.kt`

```
// TODO: Implement Google Cloud TTS
// TODO: Implement ElevenLabs TTS
// TODO: Implement OpenAI TTS
```

Impact: Only basic TTS available, premium options missing

Priority: LOW (nice-to-have features)

Recommendation: Implement incrementally based on user demand

3.5 Streaming Support

Multiple Files:

- `services/epub/EpubReaderEngine.kt` : EPUB streaming
- `services/reader/PdfReaderEngine.kt` : PDF streaming
- `services/reader/ComicReaderEngine.kt` : Comic streaming

```
// TODO: Implement streaming support for remote EPUB/PDF/Comic files
```

Impact: Can't read files directly from URLs without downloading

Priority: MEDIUM

Recommendation:

1. Implement HTTP range request support
2. Create buffering mechanism
3. Cache frequently accessed chunks

Section 4: Stub & Simplified Implementations

4.1 Jellyfin Integration (Simplified)

File: `services/integration/jellyfin/JellyfinClient.kt`

Current State: All library and item operations are stubs

```
suspend fun getLibraries(): Result<List<String>> = withContext(Dispatchers.IO) {
    Log.d(TAG, "Library listing temporarily simplified")
    Result.success(emptyList())
}

suspend fun getLibraryItems(libraryId: String): Result<List<Any>> = withCon-
text(Dispatchers.IO) {
    Log.d(TAG, "Library item listing temporarily simplified")
    Result.success(emptyList())
}

suspend fun markAsPlayed(itemId: String): Result<Unit> = withContext(Dispatchers.IO) {
    Log.d(TAG, "Mark as played temporarily simplified")
    Result.success(Unit)
}
```

Impact: Jellyfin integration non-functional

Priority: HIGH (if Jellyfin support is advertised)

Recommendation: Implement using Jellyfin SDK after fixing API compatibility issues

4.2 Plex Integration (Placeholder)

File: `services/integration/plex/PlexIntegrationService.kt`

```
// For now, return empty list as a placeholder
```

Impact: Plex integration non-functional

Priority: HIGH (if Plex support is advertised)

4.3 Emby Sync (Stub)

File: `services/integration/emby/EmbySyncService.kt`

```
// Create MediaItem stub
```

Impact: Emby integration incomplete

Priority: MEDIUM

4.4 Archive Comparison (Placeholders)

File: `services/analysis/comparison/ArchiveComparator.kt`

```
// Placeholder for Library of Congress API integration
// Placeholder for WorldCat API integration
// Placeholder for Internet Archive API integration
// Placeholder for Goodreads API integration
```

Impact: Archive comparison features non-functional

Priority: LOW (advanced feature)

4.5 Music Player Features (Simplified)

File: `services/music/AdvancedMusicPlayerService.kt`

```
// This is a placeholder for crossfade functionality
// This is a placeholder for gapless functionality
// This is a simplified version - would use actual metadata extraction
```

Impact: Advanced music player features missing

Priority: MEDIUM

Section 5: Architecture & Design Analysis

5.1 Architecture Overview

Pattern: Clean Architecture with MVVM

Layers:

```
UI Layer (Jetpack Compose)
├── ViewModels (Hilt injected)
│   └── Use Cases / Repositories
│       ├── Data Sources (Room, Retrofit, Services)
│       └── External APIs / Local Storage
```

Strengths:

-  Clear separation of concerns
-  Dependency injection with Hilt
-  Reactive state management with Kotlin Flows
-  Modern Jetpack Compose UI
-  Room database for local persistence
-  Comprehensive service layer

Concerns:

-  Large number of services (40+ files in `/services`)
-  Deep nesting in some areas
-  Potential service duplication (e.g., multiple media player services)

5.2 Package Structure

```

com.universalmedialibrary/
├── data/
│   ├── local/           # Room DAOs and entities
│   ├── models/          # Data models
│   └── migration/       # Database migrations
├── di/                  # Hilt dependency injection modules
├── services/            # Business logic layer
│   ├── analysis/        # Content analysis (OCR, classification, fingerprinting)
│   ├── audio/           # Audio playback
│   ├── comic/           # Comic processing & translation
│   ├── epub/            # EPUB reading
│   └── integration/     # External service integrations (Jellyfin, Plex, Emby, Cal-
│       ibre)
│   ├── media/           # Media scanning, playback, notifications
│   ├── playlist/        # Playlist management
│   ├── reader/          # Unified reader service
│   ├── tts/             # Text-to-speech
│   └── ...
├── ui/                  # Jetpack Compose UI
│   ├── reader/          # Reader screens
│   └── ...
└── CleverFerretApplication.kt # Application entry point

```

5.3 Service Organization Recommendations

Current Issues:

1. Multiple overlapping media services:

- UniversalMediaPlayerService
- MediaSessionManager
- MediaNotificationService
- AdvancedMusicPlayerService
- AudiobookService

1. Reader services fragmentation:

- EpubReaderEngine
- RadiumEpubService
- PdfReaderEngine
- ComicReaderEngine
- UnifiedReaderService

Recommendations:

Consolidate Media Services

```

services/media/
├── MediaPlayerService.kt    # Single service for all playback
├── MediaSessionController.kt # Media session management
├── MediaNotificationController.kt # Notification handling
└── players/
    ├── AudioPlayer.kt
    ├── VideoPlayer.kt
    └── StreamingPlayer.kt

```

Unify Reader Architecture

```

services/reader/
├── UnifiedReaderService.kt      # Main reader service
├── engines/
│   ├── EpubEngine.kt          # Powered by Radium
│   ├── PdfEngine.kt
│   └── ComicEngine.kt
└── ReaderController.kt        # State management

```

5.4 Error Handling Patterns

Current Approach: Mix of `Result<T>` and exceptions

Inconsistencies:

```

// Some services use Result
suspend fun authenticate(...): Result<String>

// Others throw exceptions
suspend fun extractCover(...): Bitmap? // Returns null on error

// Some log and return empty
suspend fun getLibraries(): Result<List<String>> {
    Log.d(TAG, "Library listing temporarily simplified")
    Result.success(emptyList()) // Silent failure
}

```

Recommendation: Standardize on sealed Result type

```

sealed class ServiceResult<out T> {
    data class Success<T>(val data: T) : ServiceResult<T>()
    data class Error(val exception: Exception, val message: String? = null) : Service-
Result<Nothing>()
    data class Loading(val progress: Float? = null) : ServiceResult<Nothing>()
}

// Usage
suspend fun authenticate(...): ServiceResult<String> {
    return try {
        val result = performAuth()
        ServiceResult.Success(result)
    } catch (e: Exception) {
        ServiceResult.Error(e, "Authentication failed: ${e.message}")
    }
}

```

5.5 State Management

Current: Mix of `StateFlow`, `MutableState`, `LiveData`

Recommendation: Standardize on `StateFlow` for ViewModels

```

class ReaderViewModel @Inject constructor() : ViewModel() {
    private val _uiState = MutableStateFlow(ReaderUiState())
    val uiState: StateFlow<ReaderUiState> = _uiState.asStateFlow()

    // Clear intent-based actions
    fun loadBook(bookId: String) { /* ... */ }
    fun navigateToChapter(index: Int) { /* ... */ }
}

// In Composable
val uiState by viewModel.uiState.collectAsState()

```

5.6 Testing Considerations

Current State: Test infrastructure present but incomplete

Missing:

- Unit tests for services
- Integration tests for database
- UI tests for critical flows

Recommendation:

1. Prioritize testing for:

- Media playback services
- Reader engines
- Database operations
- API integrations

1. Create test doubles for external dependencies:

```

// Test module
@Module
@TestInstallIn(
    components = [SingletonComponent::class],
    replaces = [NetworkModule::class]
)
object TestNetworkModule {
    @Provides
    @Singleton
    fun provideMockJellyfinClient(): JellyfinClient = MockJellyfinClient()
}

```

Section 6: Fix Implementation Priority

Phase 1: Critical Blockers (Week 1)

Goal: Get the app to compile

1. ☒ Fix Hilt version mismatch
2. ☒ Fix Jellyfin SDK initialization
3. ☒ Fix Radium Try API
4. ☒ Fix Play Services Tasks import
5. ☒ Fix EnhancedEReaderScreen Compose issues

6.  Fix EpubReaderEngine scope issue

Success Criteria: `./gradlew :CleverFerret:assembleDebug` succeeds

Phase 2: Core Functionality (Weeks 2-3)

Goal: Make primary features work

1.  Implement ExoPlayer in UniversalMediaPlayerService
2.  Complete Media3 MediaSession integration
3.  Fix media notification artwork loading
4.  Implement watch history tracking
5.  Implement progress tracking
6.  Complete Jellyfin library/item APIs
7.  Test EPUB reading end-to-end
8.  Test comic translation feature

Success Criteria: Core media playback and reading features functional

Phase 3: Integration Features (Weeks 4-5)

Goal: Complete external integrations

1.  Complete Plex integration
2.  Complete Emby integration
3.  Implement streaming support (EPUB, PDF, Comic)
4.  Add playlist duration calculations
5.  Implement rating system

Success Criteria: All advertised integrations working

Phase 4: Polish & Enhancements (Week 6+)

Goal: Improve UX and add nice-to-have features

1.  Consolidate media services architecture
2.  Standardize error handling
3.  Add premium TTS providers
4.  Implement archive comparison features
5.  Add comprehensive testing
6.  Performance optimization

Success Criteria: Production-ready quality

Section 7: Detailed Fix Code Examples

7.1 Complete Hilt Fix

File: `/build.gradle.kts` (root)

```

buildscript {
    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath("com.android.tools.build:gradle:8.9.1")
        classpath("org.jetbrains.kotlin:kotlin-gradle-plugin:2.0.20")
        // FIX: Update Hilt plugin to match library version
        classpath("com.google.dagger:hilt-android-gradle-plugin:2.57.2") // Changed
from 2.51.1
    }
}

plugins {
    id("com.android.application") version "8.9.1" apply false
    id("com.android.library") version "8.9.1" apply false
    id("org.jetbrains.kotlin.android") version "2.0.20" apply false
    id("com.google.devtools.ksp") version "2.0.20-1.0.25" apply false
    id("org.jetbrains.kotlin.plugin.serialization") version "2.0.20" apply false
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.20" apply false
}

tasks.register("clean", Delete::class) {
    delete(rootProject.layout.buildDirectory)
}

```

7.2 Complete Jellyfin Fix

File: services/integration/jellyfin/JellyfinClient.kt

```

package com.universalmedialibrary.services.integration.jellyfin

import android.util.Log
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import org.jellyfin.sdk.Jellyfin
import org.jellyfin.sdk.api.client.ApiClient
import org.jellyfin.sdk.api.client.extensions.systemApi
import org.jellyfin.sdk.api.client.extensions.userLibraryApi
import org.jellyfin.sdk.api.client.extensions.itemsApi
import org.jellyfin.sdk.model.ClientInfo
import org.jellyfin.sdk.model.DeviceInfo
import org.jellyfin.sdk.model.api.SystemInfo
import org.jellyfin.sdk.model.api.BaseItemDto
import org.jellyfin.sdk.model.api.BaseItemKind
import java.util.UUID
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Jellyfin Client for server integration
 * Updated for Jellyfin SDK 1.5.4
 */
@Singleton
class JellyfinClient @Inject constructor(
    private val okHttpClient: OkHttpClient
) {
    private val TAG = "JellyfinClient"
    private var jellyfin: Jellyfin? = null
    private var apiClient: ApiClient? = null
    private var currentUserId: String? = null

    /**
     * Initialize connection to Jellyfin server
     * FIXED: Updated for SDK 1.5.4 API
     */
    fun initialize(serverUrl: String, apiKey: String? = null) {
        try {
            // Create Jellyfin instance with client info
            val clientInfo = ClientInfo(
                name = "CleverFerret",
                version = "1.0.0"
            )

            val deviceInfo = DeviceInfo(
                id = UUID.randomUUID().toString(),
                name = "Android Device"
            )

            // Create Jellyfin instance using builder lambda
            jellyfin = Jellyfin {
                this.clientInfo = clientInfo
                this.deviceInfo = deviceInfo
            }

            // Create API client
            apiClient = if (apiKey != null) {
                jellyfin?.createApi(
                    baseUrl = serverUrl,
                    accessToken = apiKey
                )
            }
        }
    }
}

```

```

        } else {
            jellyfin?.createApi(baseUrl = serverUrl)
        }

        Log.d(TAG, "Jellyfin client initialized successfully")
    } catch (e: Exception) {
        Log.e(TAG, "Failed to initialize: ${e.message}", e)
    }
}

/**
 * Authenticate with API key
 */
suspend fun authenticate(serverUrl: String, apiKey: String): Result<String> =
    withContext(Dispatchers.IO) {
        try {
            initialize(serverUrl, apiKey)

            val client = apiClient ?: return@withContext Result.failure(
                IllegalStateException("API client not initialized")
            )

            // Test the connection
            val response by client.systemApi.getSystemInfo()
            Log.d(TAG, "Authentication successful: ${response.serverName}")

            Result.success(apiKey)
        } catch (e: Exception) {
            Log.e(TAG, "Authentication failed", e)
            Result.failure(e)
        }
    }

/**
 * Get server information
 */
suspend fun getServerInfo(): Result<SystemInfo> = withContext(Dispatchers.IO) {
    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val response by client.systemApi.getSystemInfo()
        Result.success(response)
    } catch (e: Exception) {
        Log.e(TAG, "Failed to get server info", e)
        Result.failure(e)
    }
}

/**
 * Set user ID for operations
 */
fun setUserId(userId: String) {
    currentUserId = userId
}

/**
 * Get libraries
 * IMPLEMENTED: Full functionality
 */
suspend fun getLibraries(): Result<List<BaseItemDto>> =
    withContext(Dispatchers.IO) {

```

```

    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val userId = currentUserId ?: return@withContext Result.failure(
            IllegalStateException("User ID not set")
        )

        val response by client.userLibraryApi.getUserViews(userId = UUID.fromString(userId))
        Result.success(response.items.orEmpty())
    } catch (e: Exception) {
        Log.e(TAG, "Failed to get libraries", e)
        Result.failure(e)
    }
}

/**
 * Get items from library
 * IMPLEMENTED: Full functionality
 */
suspend fun getLibraryItems(
    libraryId: String,
    itemType: BaseItemKind? = null
): Result<List<BaseItemDto>> = withContext(Dispatchers.IO) {
    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val userId = currentUserId ?: return@withContext Result.failure(
            IllegalStateException("User ID not set")
        )

        val response by client.itemsApi.getItems(
            userId = UUID.fromString(userId),
            parentId = UUID.fromString(libraryId),
            includeItemTypes = itemType?.let { listOf(it) }
        )

        Result.success(response.items.orEmpty())
    } catch (e: Exception) {
        Log.e(TAG, "Failed to get library items", e)
        Result.failure(e)
    }
}

/**
 * Mark item as played
 * IMPLEMENTED: Full functionality
 */
suspend fun markAsPlayed(itemId: String): Result<Unit> = withContext(Dispatchers.IO) {
    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val userId = currentUserId ?: return@withContext Result.failure(
            IllegalStateException("User ID not set")
        )
    }
}

```



```

        client.userLibraryApi.markPlayedItem(
            userId = UUID.fromString(userId),
            itemId = UUID.fromString(itemId)
        )

        Result.success(Unit)
    } catch (e: Exception) {
        Log.e(TAG, "Failed to mark as played", e)
        Result.failure(e)
    }
}

/**
 * Mark item as unplayed
 * IMPLEMENTED: Full functionality
 */
suspend fun markAsUnplayed(itemId: String): Result<Unit> = withCon-
text(Dispatchers.IO) {
    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val userId = currentUserId ?: return@withContext Result.failure(
            IllegalStateException("User ID not set")
        )

        client.userLibraryApi.markUnplayedItem(
            userId = UUID.fromString(userId),
            itemId = UUID.fromString(itemId)
        )

        Result.success(Unit)
    } catch (e: Exception) {
        Log.e(TAG, "Failed to mark as unplayed", e)
        Result.failure(e)
    }
}

/**
 * Update playback position
 * IMPLEMENTED: Full functionality
 */
suspend fun updatePlaybackPosition(
    itemId: String,
    positionTicks: Long
): Result<Unit> = withContext(Dispatchers.IO) {
    try {
        val client = apiClient ?: return@withContext Result.failure(
            IllegalStateException("API client not initialized")
        )

        val userId = currentUserId ?: return@withContext Result.failure(
            IllegalStateException("User ID not set")
        )

        client.itemsApi.markPlayedItem(
            userId = UUID.fromString(userId),
            itemId = UUID.fromString(itemId),
            datePlayed = System.currentTimeMillis()
        )

        Result.success(Unit)
    }

```

```
    } catch (e: Exception) {  
        Log.e(TAG, "Failed to update playback position", e)  
        Result.failure(e)  
    }  
}
```

7.3 Complete Radium Fix

File: services/epub/RadiumEpubService.kt

```

suspend fun extractCover(epubPath: String): Bitmap? = withContext(Dispatchers.IO) {
    try {
        val file = File(epubPath)
        if (!file.exists()) {
            Log.e(TAG, "EPUB file not found: $epubPath")
            return@withContext null
        }

        // Retrieve the asset from the file
        // FIXED: Handle Try result with proper type parameters
        val asset = when (val assetResult = assetRetriever.retrieve(file)) {
            is Try.Success -> assetResult.value
            is Try.Failure -> {
                Log.e(TAG, "Failed to retrieve asset: ${assetResult.value}")
                return@withContext null
            }
        }

        // Parse EPUB
        // FIXED: Handle Try result properly
        val publication = when (val parseResult = epubParser.parse(asset)) {
            is Try.Success -> parseResult.value.build()
            is Try.Failure -> {
                Log.e(TAG, "Failed to parse EPUB: ${parseResult.value}")
                return@withContext null
            }
        }

        val coverBitmap = extractCoverFromPublication(publication)
        publication.close()
        coverBitmap
    } catch (e: Exception) {
        Log.e(TAG, "Error extracting cover: ${e.message}", e)
        null
    }
}

suspend fun openEpub(epubPath: String): Publication? = withContext(Dispatchers.IO) {
    try {
        val file = File(epubPath)
        if (!file.exists()) {
            Log.e(TAG, "EPUB file not found: $epubPath")
            return@withContext null
        }

        // FIXED: Same pattern for opening
        val asset = when (val assetResult = assetRetriever.retrieve(file)) {
            is Try.Success -> assetResult.value
            is Try.Failure -> {
                Log.e(TAG, "Failed to retrieve asset: ${assetResult.value}")
                return@withContext null
            }
        }

        val publication = when (val parseResult = epubParser.parse(asset)) {
            is Try.Success -> parseResult.value.build()
            is Try.Failure -> {
                Log.e(TAG, "Failed to parse EPUB: ${parseResult.value}")
                return@withContext null
            }
        }
    }
}

```

```
        publication
    } catch (e: Exception) {
        Log.e(TAG, "Error opening EPUB: ${e.message}", e)
        null
    }
}
```

7.4 Complete ComicProcessor Fix

File: services/comic/ComicProcessorRepository.kt

```

package com.universalmedialibrary.services.comic

import android.graphics.Bitmap
import android.graphics.BitmapFactory
import com.google.ai.client.generativeai.GenerativeModel
import com.google.ai.client.generativeai.type.*
import com.google.android.gms.tasks.Task
import com.google.mlkit.common.model.DownloadConditions
import com.google.mlkit.nl.translate.TranslateLanguage
import com.google.mlkit.nl.translate.Translation
import com.google.mlkit.nl.translate.TranslatorOptions
import com.universalmedialibrary.data.local.dao.ComicTranslationCacheDao
import com.universalmedialibrary.data.local.entity.ComicPageResponse
import com.universalmedialibrary.data.local.entity.ComicTranslationCache
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import kotlinx.serialization.json.Json
import java.util.Locale
import javax.inject.Inject
import kotlin.coroutines.resume
import kotlin.coroutines.resumeWithException
import kotlin.coroutines.suspendCoroutine

/**
 * Comic Processor Repository
 * FIXED: Added custom await extension for Google Play Services Tasks
 */

// ADDED: Custom await extension function
private suspend fun <T> Task<T>.await(): T = suspendCoroutine { continuation ->
    addOnSuccessListener { result ->
        continuation.resume(result)
    }
    addOnFailureListener { exception ->
        continuation.resumeWithException(exception)
    }
    addOnCanceledListener {
        continuation.resumeWithException(CancellationException("Task was cancelled"))
    }
}

class CancellationException(message: String) : Exception(message)

@Singleton
class ComicProcessorRepository @Inject constructor(
    private val translationCacheDao: ComicTranslationCacheDao,
    private val generativeModel: GenerativeModel
) {
    private val TAG = "ComicProcessor"

    // ... rest of the class ...

    /**
     * Translate comic page response to target language
     * FIXED: Uses custom await extension
     */
    private suspend fun translateResponse(
        response: ComicPageResponse,
        targetLanguage: String
    ): ComicPageResponse = withContext(Dispatchers.IO) {
        val mlKitLanguage = when (targetLanguage.lowercase()) {
            "english", "en" -> TranslateLanguage.ENGLISH

```

```

        "spanish", "es" -> TranslateLanguage.SPANISH
        "french", "fr" -> TranslateLanguage.FRENCH
        "german", "de" -> TranslateLanguage.GERMAN
        "japanese", "ja" -> TranslateLanguage.JAPANESE
        "korean", "ko" -> TranslateLanguage.KOREAN
        "chinese", "zh" -> TranslateLanguage.CHINESE
        else -> TranslateLanguage.ENGLISH
    }

    val options = TranslatorOptions.Builder()
        .setSourceLanguage(TranslateLanguage.ENGLISH)
        .setTargetLanguage(mlKitLanguage)
        .build()

    val translator = Translation.getClient(options)

    // Download model if needed
    val conditions = DownloadConditions.Builder()
        .requireWifi()
        .build()

    try {
        // FIXED: Using custom await extension
        translator.downloadModelIfNeeded(conditions).await()

        val translatedPanels = response.panels.map { panel ->
            val translatedElements = panel.textElements.map { textElement ->
                val translatedText = performTranslation(translator, textElement.translatedText)
                textElement.copy(translatedText = translatedText)
            }
            panel.copy(textElements = translatedElements)
        }

        translator.close()
        response.copy(panels = translatedPanels)
    } catch (e: Exception) {
        translator.close()
        // Return original response if translation fails
        response
    }
}

/**
 * Perform translation using ML Kit translator
 * FIXED: Using custom await extension
 */
private suspend fun performTranslation(
    translator: com.google.mlkit.nl.translate.Translator,
    text: String
): String {
    return try {
        // FIXED: Using custom await extension
        translator.translate(text).await()
    } catch (e: Exception) {
        // Return original text on failure
        text
    }
}

/**
 * Process comic page image with Gemini
 * FIXED: Type conversion for bitmap

```

```

*/
suspend fun processComicPage(
    pageImage: Bitmap,
    targetLanguage: String = "English"
): ComicPageResponse? = withContext(Dispatchers.IO) {
    try {
        // Convert bitmap to byte array for Gemini
        val stream = java.io.ByteArrayOutputStream()
        pageImage.compress(Bitmap.CompressFormat.PNG, 100, stream)
        val imageBytes = stream.toByteArray()

        val prompt = """
            Analyze this comic page and provide:
            1. Panel boundaries (x, y, width, height)
            2. Text bubbles and their locations
            3. Extracted text from each bubble
            4. Reading order

            Return as JSON with structure:
            {
              "panels": [
                {
                  "id": "panel_1",
                  "bounds": {"x": 0, "y": 0, "width": 100, "height": 100},
                  "textElements": [
                    {
                      "text": "Original text",
                      "bounds": {"x": 10, "y": 10, "width": 50, "height": 20}
                    }
                  ]
                }
              ]
            }
            """.trimIndent()

        val content = content {
            image(imageBytes)
            text(prompt)
        }

        val response = generativeModel.generateContent(content)
        val jsonText = response.text ?: return@withContext null

        // Parse JSON response
        val comicResponse = Json.decodeFromString<ComicPageResponse>(jsonText)

        // Translate if needed
        if (targetLanguage.lowercase() != "english") {
            translateResponse(comicResponse, targetLanguage)
        } else {
            comicResponse
        }
    } catch (e: Exception) {
        Log.e(TAG, "Failed to process comic page", e)
        null
    }
}
}

```

Section 8: Testing & Validation Plan

8.1 Build Validation

Checklist after fixes:

```
# 1. Clean build
./gradlew clean

# 2. Verify dependencies
./gradlew :CleverFerret:dependencies

# 3. Compile Kotlin
./gradlew :CleverFerret:compileDebugKotlin

# 4. Run KSP processors
./gradlew :CleverFerret:kspDebugKotlin

# 5. Full build
./gradlew :CleverFerret:assembleDebug

# 6. Check for lint issues
./gradlew :CleverFerret:lintDebug
```

8.2 Runtime Testing

Critical User Flows:

1. ☒ App launches successfully
2. ☒ Media library scans files
3. ☒ EPUB reader opens and displays book
4. ☒ Media player plays audio/video
5. ☒ Jellyfin connection works
6. ☒ Comic translation processes page

Testing Script:

```
# Build and install
./gradlew installDebug

# Run instrumentation tests
./gradlew connectedDebugAndroidTest

# Generate test coverage
./gradlew jacocoTestReport
```

8.3 Integration Testing

Jellyfin Integration:


```

@Test
fun testJellyfinConnection() = runBlocking {
    val client = JellyfinClient(okHttpClient)
    client.initialize("http://demo.jellyfin.org", "test-api-key")

    val result = client.getServerInfo()
    assertTrue(result.isSuccess)
}

```

EPUB Reading:

```

@Test
fun testEpubOpening() = runBlocking {
    val service = RadiumEpubService(context)
    val publication = service.openEpub("/path/to/test.epub")

    assertNotNull(publication)
    assertTrue(publication.readingOrder.isNotEmpty())
}

```

Section 9: Documentation Updates Needed

9.1 README Updates

Current Status: Likely outdated with old API references

Required Updates:

1. Update Jellyfin SDK version to 1.5.4
2. Document Radium 3.1.2 integration
3. Add troubleshooting section for common build errors
4. Update dependency versions table
5. Add architecture diagram

9.2 API Documentation

Files to Create:

- docs/JELLYFIN_INTEGRATION.md - Jellyfin setup and usage
- docs/READIUM_INTEGRATION.md - EPUB reading implementation
- docs/COMIC_TRANSLATION.md - Comic processing workflow
- docs/HILT_SETUP.md - DI configuration guide

9.3 Code Comments

Areas Needing Better Documentation:

- Complex Compose state management in EnhancedEReaderScreen
 - Media session lifecycle in media services
 - Database migration strategies
 - Comic translation caching logic
-

Section 10: Performance Considerations

10.1 Current Performance Risks

Database Operations:

- Room queries without indexing on frequently searched columns
- Potential N+1 query problems in playlist managers

Recommendation:

```
// Add indices to frequently queried columns
@Entity(
    tableName = "media_items",
    indices = [
        Index(value = ["type"]),
        Index(value = ["path"]),
        Index(value = ["addedDate"])
    ]
)
data class MediaItem(...)
```

Memory Management:

- Large bitmap operations in ComicProcessor
- EPUB content loaded entirely into memory

Recommendation:

```
// Stream EPUB content instead of loading all at once
fun streamChapter(chapterIndex: Int): Flow<String> = flow {
    val chapter = getChapter(chapterIndex)
    val reader = chapter.content.reader()

    reader.useLines { lines ->
        lines.forEach { emit(it) }
    }
}
```

10.2 Background Processing

Current: Multiple services running simultaneously

Optimization:

```
// Use WorkManager for non-urgent background tasks
class MediaScanWorker(context: Context, params: WorkerParameters) : CoroutineWorker(
    context, params) {
    override suspend fun doWork(): Result {
        // Scan media with constraints
        return Result.success()
    }
}

// Schedule with constraints
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.UNMETERED)
    .setRequiresBatteryNotLow(true)
    .build()

val scanRequest = PeriodicWorkRequestBuilder<MediaScanWorker>(1, TimeUnit.DAYS)
    .setConstraints(constraints)
    .build()
```

Conclusion

The CleverFerret Android app is an ambitious project with solid architectural foundations but currently blocked by critical compilation errors. The primary issues are:

1. **Hilt version mismatch** - Quick fix by aligning plugin and library versions
2. **Jellyfin SDK API changes** - Requires updating initialization code
3. **Radium Try type handling** - Needs proper when expression with type parameters
4. **Play Services Tasks import** - May require custom await extension
5. **Compose scope issues** - Needs refactoring of UI callbacks

Estimated Fix Time:

- Phase 1 (Critical Blockers): 2-3 days
- Phase 2 (Core Functionality): 1-2 weeks
- Phase 3 (Integrations): 1-2 weeks
- Phase 4 (Polish): Ongoing

Risk Assessment:

-  **HIGH**: Without fixes, app cannot build
-  **MEDIUM**: Incomplete features may confuse users
-  **LOW**: Architecture is sound, just needs implementation completion

Recommended Next Steps:

1. Apply Hilt version fix immediately
 2. Run diagnostic build to verify dependency resolution
 3. Fix Jellyfin and Radium API compatibility
 4. Add custom await extension for Play Services
 5. Refactor Compose UI issues
 6. Create comprehensive test suite
 7. Document all integrations
-

Document Version: 1.0

Last Updated: October 14, 2025

Contributors: Analysis Team